

RPGBot — Complete Cross-Platform Browser, Termux & Android Support Plan

Project

Repository: "CountKrampus/rpgbot"

Project: Eclipse RPG Automation / RPGBot

Primary desktop platform: Windows 11

New mobile platform: Android + Termux

Target browsers

- Brave
- Google Chrome
- Chromium
- Automatic browser detection

Target environments

- Windows 11
- Android + Termux
- Android browsers where technically supported through Selenium/CDP/ADB

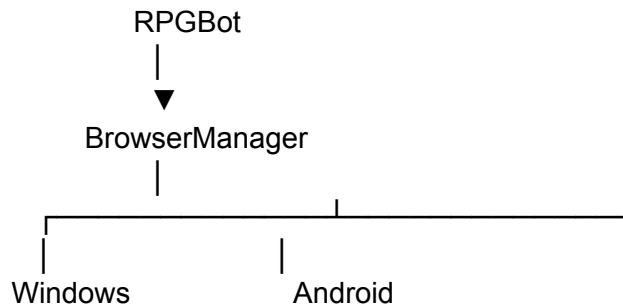
1. Project Vision

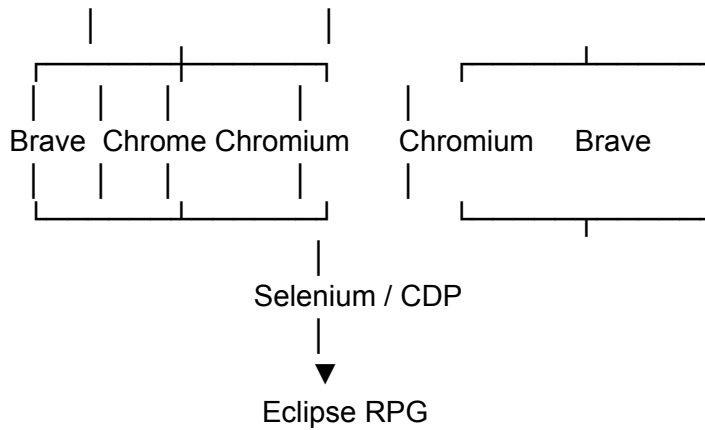
Transform RPGBot into a single cross-platform Eclipse RPG automation system capable of running on both Windows and Android without maintaining separate versions of the bot.

The core Eclipse RPG automation should remain platform-independent.

The browser and operating system should be implementation details handled by a dedicated browser/platform abstraction layer.

The final architecture should allow:





The user should ultimately be able to install the same repository and choose:

Windows + Brave
 Windows + Chrome
 Windows + Chromium

Android + Termux + Chromium
 Android + Termux + Brave

without maintaining different Eclipse automation code.

2. Fundamental Design Principle

One bot. One repository. Multiple browser/platform backends.

Do not create:

rpgbot-windows/
 rpgbot-android/
 rpgbot-brave/

Instead maintain:

CountKrampus/rpgbot

with a centralized browser abstraction.

The existing modules should continue to receive a normal Selenium WebDriver.

For example:

```
driver = create_driver()
```

Then:

```
training.py  
capture.py  
mining.py  
shop.py  
login.py  
menus/
```



Selenium driver



BrowserManager



├── Windows

└── Android / Termux

3. Current RPSBot Must Be Audited First

Before modifying browser startup, perform a complete browser architecture audit.

Identify every location where RPSBot:

- Creates a WebDriver
- Creates ChromeDriver
- Creates BraveDriver
- Creates Chromium options
- Sets browser executable paths
- Sets driver executable paths
- Uses "webdriver.Chrome()"
- Uses Selenium Manager
- Uses subprocesses to start browsers
- Closes browsers
- Creates browser profiles
- Uses cookies
- Uses local storage
- Uses user-data directories
- Uses Windows-specific paths

- Uses Windows-specific commands
- Uses hardcoded ".exe" files
- Uses environment variables
- Uses browser-specific command-line arguments

Also inspect:

config.py
settings.py
settings.json
login.py
main/startup code
update system
restart system
requirements.txt

Audit rule

Do not begin rewriting the automation modules until all WebDriver creation points are known.

4. Browser Abstraction Layer

Create a centralized browser subsystem.

Potential structure:

browser/
 __init__.py
 manager.py
 platform.py
 windows.py
 android.py
 chromium.py
 chrome.py
 brave.py

The exact structure should be adapted to the existing repository.

Do not restructure unrelated parts of RPGBot merely to match this example.

5. BrowserManager

The BrowserManager becomes the central point for browser creation.

Preferred interface:

```
driver = BrowserManager.create()
```

or:

```
driver = create_driver()
```

Optional explicit configuration:

```
driver = BrowserManager.create(  
    browser="brave",  
    platform="android"  
)
```

However, normal RPGBot modules should not need to specify the browser.

They should simply receive:

```
driver
```

```
---
```

6. Platform Detection

RPGBot should automatically detect the execution environment.

Possible values:

```
windows  
android  
linux  
unknown
```

Termux should be recognized as:

```
Android / Termux
```

rather than treated as an ordinary desktop Linux installation.

Conceptual API:

```
platform = detect_platform()
```

The browser manager then selects the correct backend.

7. Browser Detection

Support:

auto

brave

chrome

chromium

The selected browser should be verified before startup.

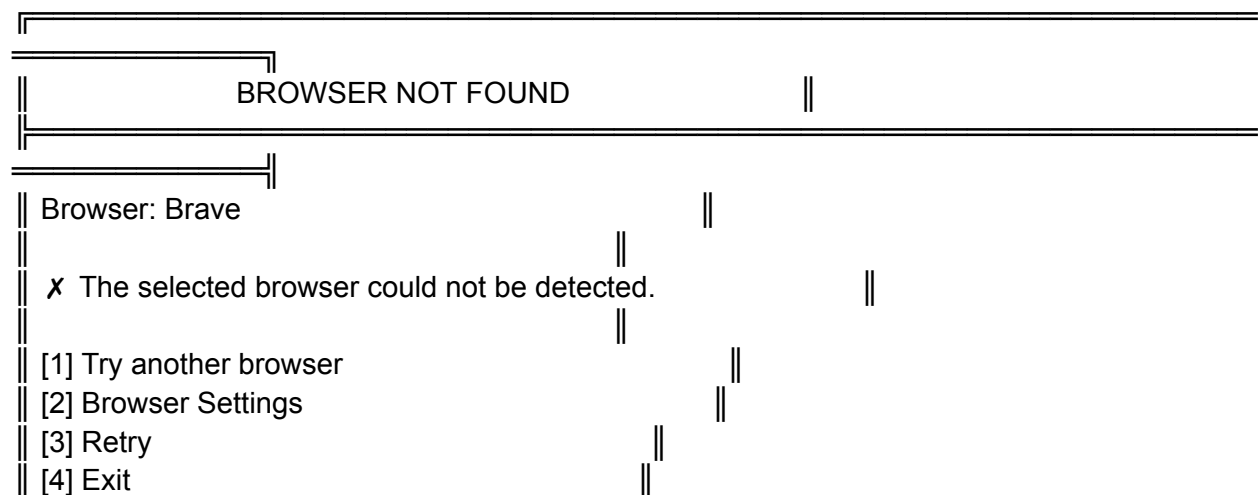
Example:

Detecting browser...

- ✓ Browser detected
- ✓ Browser version detected
- ✓ Automation interface available

Starting browser...

If a requested browser is unavailable:



Automatic fallback must only happen when the user has selected automatic detection or explicitly enabled fallback.

8. Windows Browser Support

Windows remains the first compatibility target because it is the existing production environment.

Support:

Windows + Brave
Windows + Chrome
Windows + Chromium
Windows + Auto Detect

The Windows backend should preserve the current RPSBot behavior wherever possible.

9. Windows Browser Configuration

Browser configuration should be discoverable rather than relying on hardcoded assumptions.

Avoid permanently assuming paths such as:

C:\Program Files\Google\Chrome\
C:\Program Files\BraveSoftware\

Instead:

1. Check known installation locations.
2. Check environment/PATH where appropriate.
3. Check configured paths.
4. Validate the executable.
5. Determine the browser version.
6. Configure Selenium accordingly.

10. Android + Termux

The primary mobile target is:

Android
↓
Termux
↓
Python
↓
RPGBot

The first browser target should be Chromium.

Initial proof of concept:

Termux
↓
Python
↓
Selenium
↓
Chromium
↓
Eclipse RPG

Do not integrate Android into RPGBot until this standalone configuration is proven.

11. Termux Chromium Proof of Concept

Create a minimal independent test program.

The test must establish that:

Python
↓
Selenium
↓
Chromium

works correctly inside Termux.

Test:

- Python startup
- Selenium import
- Browser discovery
- WebDriver startup
- Page navigation
- JavaScript
- CSS selectors
- XPath
- Element lookup
- Clicking
- Text entry
- WebDriverWait
- JavaScript execution
- Page refresh
- Browser shutdown

Then test Eclipse RPG itself.

12. Android Brave Support

Brave should be treated as a separate Android backend.

Do not assume Android Brave behaves exactly like desktop Brave.

The implementation must determine:

- Actual Brave package name
- Available debugging interface
- Remote debugging availability
- DevTools/CDP connectivity
- Whether the browser can be controlled while running normally
- Whether a dedicated browser instance can be launched
- Whether an existing browser can be attached to

Do not hardcode an unverified Android package identifier.

13. Android Chrome Support

Chrome on Android should remain an optional backend.

The architecture should allow:

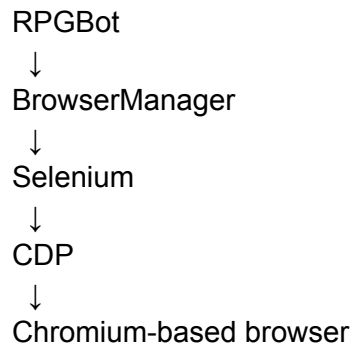
Android + Chrome

where technically practical, but it should not block the project if Chromium and Brave are the primary supported mobile browsers.

14. Selenium + CDP Architecture

The browser layer should support Chromium DevTools Protocol where necessary.

Conceptually:



CDP is especially important for Android browser automation and browser attachment scenarios.

15. Browser Connection Modes

Support these logical modes:

- [1] Launch Browser
- [2] Connect to Existing Browser
- [3] Android / CDP
- [4] Auto

Launch Browser

RPGBot starts the browser itself.

Existing Browser

RPGBot attaches to an already-running browser where supported.

Android / CDP

RPGBot uses the Android-specific debugging/automation path.

Auto

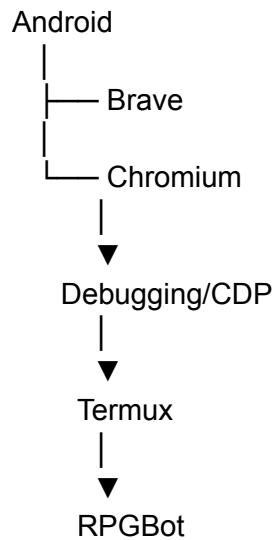
RPGBot determines the correct approach.

16. Existing Browser Attachment

Where supported, allow RPGBot to connect to a browser that is already running.

This is especially useful on Android.

Conceptual architecture:



This can avoid forcing RPGBot to directly launch the Android browser process.

17. Browser Settings

Integrate browser settings into the existing "settings.py" system.

Do not create a second configuration framework.

Potential configuration:

```
{
  "browser": {
    "name": "auto",
    "platform": "auto",
    "mode": "auto",
    "headless": false,
    "use_remote_debugging": false,
    "debugger_address": null,
    "keep_browser_open": false,
    "allow_fallback": false
  }
}
```

The exact schema must be adapted to the existing settings architecture.

18. Backward Compatibility With settings.json

Existing settings files must continue working.

If browser settings don't exist:

use defaults

Do not require users to manually rewrite their existing "settings.json".

Example default:

Browser: Auto

Platform: Auto

Mode: Auto

19. Browser Settings Menu

Add browser configuration to the existing RPGBot settings system.

Use the established continuous ANSI box style.

Example:

BROWSER SETTINGS	
Configure browser automation	
Browser:	Auto Detect
Platform:	Automatic
Mode:	Automatic
Headless:	Disabled
Remote Debugging:	Disabled
Keep Browser Open:	Disabled
Fallback:	Disabled

[1] Select Browser
[2] Select Platform
[3] Select Browser Mode
[4] Toggle Headless Mode
[5] Toggle Remote Debugging
[6] Configure Debugger Address
[7] Toggle Keep Browser Open
[8] Toggle Browser Fallback
[9] Test Browser
[0] Back

20. Browser Selection Menu

Possible choices:

- [1] Brave
- [2] Chrome
- [3] Chromium
- [4] Auto Detect
- [5] Back

On Android, unavailable browsers should be hidden or clearly marked unavailable.

21. Platform Selection

Possible choices:

- [1] Windows
- [2] Android / Termux
- [3] Auto Detect
- [4] Back

Normally:

Platform = Auto

should be preferred.

22. Browser Mode Selection

Possible choices:

- [1] Launch Browser
- [2] Existing Browser
- [3] CDP / Remote Debugging
- [4] Auto

The actual supported combinations should be validated by the browser backend.

23. BrowserManager Responsibilities

The BrowserManager should handle:

- Platform detection
- Browser detection
- Browser selection
- Browser executable discovery
- Browser package discovery
- Selenium options
- Driver configuration

- CDP configuration
- Remote debugging
- Browser profiles
- Browser startup
- Existing-browser attachment
- Browser shutdown
- Startup errors
- Diagnostics
- Browser fallback

It should not contain Eclipse-specific automation logic.

24. Eclipse Automation Responsibilities

Existing modules remain responsible for Eclipse RPG.

Examples:

login.py
training.py
capture.py
mining.py
shop.py

They should continue operating against:

driver

without caring about the browser.

Avoid:

if android:

...

elif brave:

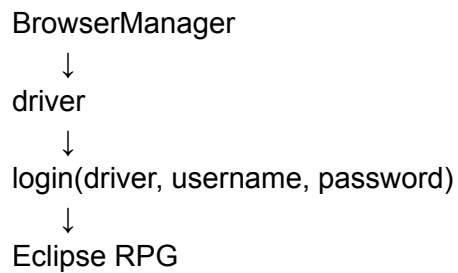
...

inside Eclipse automation modules unless the actual website interaction genuinely requires platform-specific behavior.

25. Login Architecture

The login system should remain shared.

Desired flow:



There should not be:

Windows login
Android login
Brave login
Chrome login

The same login implementation should be used whenever possible.

26. Browser Profiles

Audit whether RPGBot currently relies on:

- Cookies
- Local storage
- Session storage
- Cached credentials
- Browser profiles
- Persistent login sessions

If persistent sessions are required, implement a dedicated RPGBot browser profile.

Example:

RPGBot Profile

- Cookies
- Local Storage
- Session Data

└─ Browser Preferences

Never overwrite the user's normal personal browser profile.

27. Session Persistence Testing

Test:

Start RPGBot



Login



Close RPGBot



Restart RPGBot



Is session retained?

Perform this test separately on:

Windows + Brave

Windows + Chrome

Windows + Chromium

Android + Chromium

Android + Brave

28. Portable Paths

Perform a complete path audit.

Remove unnecessary assumptions such as:

F:\New folder\eclipse

from application logic.

Use:

from pathlib import Path

and dynamically determine:

- Project root
- Configuration directory
- Settings location
- Log directory
- Browser profile
- Cache
- Temporary files
- Update files

The same source should work in:

Windows:

F:\New folder\eclipse

Android:

~/rpgbot

29. File Storage

Identify all persistent RPGBot files.

Potential categories:

Configuration

Logs

Browser profiles

Cookies

Session data

Cache

Temporary files

Update data

Backups

Use platform-appropriate locations.

30. Dependency Management

Audit:

requirements.txt

and all Python dependencies.

Separate:

shared dependencies

from:

platform-specific dependencies

Do not force Windows users to install unnecessary Android dependencies.

31. Termux Setup System

Create dedicated Termux documentation.

Potential files:

docs/

TERMUX_SETUP.md

ANDROID_BROWSER_SETUP.md

BROWSER_CONFIGURATION.md

Potential setup script:

scripts/

setup_termux.sh

32. Termux Setup Script

The setup script should:

1. Verify Termux.
2. Verify Android environment.
3. Verify Python.
4. Verify pip.

5. Install shared dependencies.
6. Install required browser automation dependencies.
7. Detect Chromium.
8. Detect Brave.
9. Verify Selenium.
10. Verify browser connectivity.
11. Run a browser test.
12. Provide clear troubleshooting information.

Do not assume every Android device has identical browser installations.

33. Browser Diagnostics

Add a dedicated diagnostic system.

Example:

BROWSER TEST			
Platform: Android / Termux			
Browser: Chromium			
Mode: CDP			
✓ Python			
✓ Selenium			
✓ Browser detected			
✓ Browser version detected			
✓ Driver connection			
✓ CDP connection			
✓ Navigation			
✓ JavaScript			
✓ Element lookup			
✓ WebDriverWait			
✓ Browser interaction			
Browser test successful.			

34. Error Handling

Browser errors should be translated into useful user-facing messages.

Do not dump large Selenium tracebacks into the normal UI.

Example:

```

|-----|
|         |
|         | BROWSER STARTUP ERROR         |
|         |                               |
|-----|
|         |
| Browser: Brave                               ||
| Platform: Android / Termux                 ||
|                                             ||
| x Unable to connect to the browser.         ||
|                                             ||
| Possible causes:                           ||
| • Browser is not installed                  ||
| • Remote debugging is unavailable          ||
| • Browser is already controlled by another session ||
| • Driver connection failed                 ||
| • Browser version is incompatible          ||
|                                             ||
| [1] Retry                                ||
| [2] Change Browser                       ||
| [3] Browser Settings                     ||
| [4] Diagnostics                          ||
| [5] Exit                                ||
|         |
|-----|
|         |
|-----|

```

Detailed exception information should go to the log.

35. Browser Recovery

Implement recovery after the basic system is stable.

Potential flow:

Browser connection lost.

Attempting recovery...

[1/3] Reconnecting...

[2/3] Reconnecting...

[3/3] Reconnecting...

✓ Browser connection restored.

If recovery fails:

Browser connection could not be restored.

Saving current bot state...

Returning safely...

Recovery behavior should be aware of what automation module was running.

36. Network Recovery

Eclipse automation is internet-dependent.

Implement appropriate handling for:

- Temporary connection loss
- Page timeout
- DNS failure
- Browser network errors

Server timeout

Selenium timeout

The bot should distinguish:

Browser failure

from:

Eclipse RPG/server/network failure

37. Android Background Operation

Termux introduces issues that don't exist on Windows.

Investigate:

Android process restrictions

Battery optimization

Background execution limits

Termux process termination

Browser suspension

Screen-off behavior

Memory pressure

Network suspension

Long-running process stability

These should be documented separately from browser configuration.

38. Long-Running Termux Test

Perform an extended test:

Start RPGBot



Login



Run automation



Leave Termux running



Monitor



Verify browser



Verify bot

Test:

30 minutes

1 hour

Several hours

Overnight if practical

Record:

Memory usage

CPU usage

Browser disconnects

Selenium errors

Network errors

Android process termination

39. Auto-Update Compatibility

The existing RPGBot auto-update system must be audited for cross-platform behavior.

Identify assumptions involving:

.exe

PowerShell

Windows paths

Windows process management

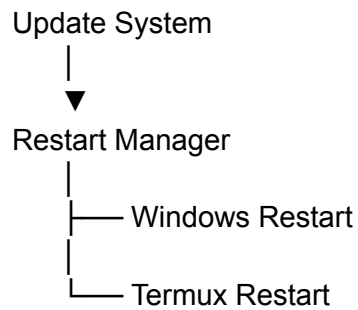
Windows shell commands

The update mechanism should remain shared where possible.

40. Platform-Specific Restart

Restart behavior should be abstracted.

Architecture:



Windows and Termux may require completely different restart mechanisms.

Do not put those differences into the main updater.

41. Auto-Update Safety

Before automatically restarting:

Save settings.

Save bot state if applicable.

Close browser cleanly.

Apply update.

Start the new version.

Verify startup.

Restore/reconnect browser if appropriate.

If restart fails, provide recovery instructions rather than repeatedly restarting.

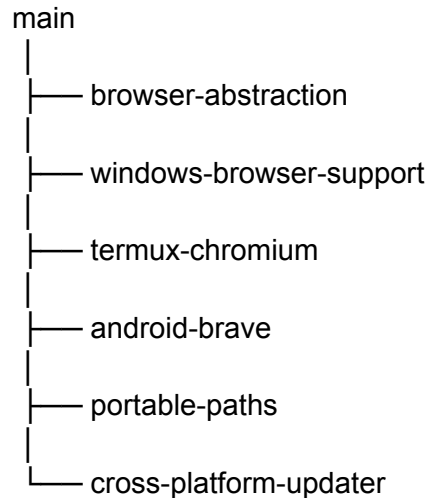
42. GitHub / Repository Updates

The cross-platform architecture should work with:

CountKrampus/rpgbot

and maintain normal Git workflows.

Recommended development flow:



Do not implement every feature in one enormous change.

43. Commit Strategy

Prefer small logical commits.

Example:

- Add browser manager abstraction
- Add Windows browser backend
- Add browser settings
- Add browser diagnostics
- Add Chromium detection
- Add Termux browser proof of concept
- Add Android Chromium backend
- Add Android Brave backend
- Add portable path handling
- Add browser recovery
- Make updater platform-aware
- Add Termux documentation

44. Rollback Strategy

Before major modifications:

```
git status
git add .
git commit -m "Checkpoint before cross-platform browser work"
git push
```

Maintain known-good commits.

Never perform a repository-wide migration without a recoverable checkpoint.

45. Development Order

Phase 1 — Audit

No behavioral changes.

Deliver:

Browser architecture audit

Phase 2 — Centralize Windows Browser Startup

Move the existing Windows WebDriver creation behind BrowserManager.

Requirement:

Windows RPSBot must behave exactly as it did before the abstraction.

Phase 3 — Windows Browser Support

Implement:

Windows + Brave

Windows + Chrome

Windows + Chromium

Windows + Auto Detect

Phase 4 — Browser Settings

Integrate:

Browser

Platform

Mode

Headless

Remote debugging

Debugger address

Browser fallback

into the existing settings system.

Phase 5 — Browser Diagnostics

Implement:

Browser Test

before Android support.

Phase 6 — Termux Proof of Concept

Independently establish:

Termux

↓

Python

↓

Selenium

↓

Chromium

Phase 7 — Android Chromium Backend

Integrate the proven Termux configuration into BrowserManager.

Target:

Android + Termux + Chromium

Phase 8 — Android Brave

Implement and test:

Android + Termux + Brave

using the actual browser/package/debugging configuration found on the device.

Phase 9 — CDP / Existing Browser Support

Implement:

CDP

Remote debugging

Existing browser attachment

where supported.

Phase 10 — Portable Paths

Complete the path migration.

Phase 11 — Session/Profile Support

Implement persistent browser profiles where necessary.

Phase 12 — Browser Recovery

Add:

Browser reconnect

Network recovery

Graceful failure

Phase 13 — Update/Restart Compatibility

Make auto-update and restart behavior platform-aware.

Phase 14 — Extended Android Testing

Perform long-running Termux tests.

Phase 15 — Documentation

Finalize:

TERMUX_SETUP.md

ANDROID_BROWSER_SETUP.md

BROWSER_CONFIGURATION.md

46. Testing Matrix

Platform

Browser

Mode

Priority

Windows

Brave

Launch

High

Windows

Chrome

Launch

High

Windows

Chromium

Launch

Medium

Windows

Auto

Auto

High

Android/Termux

Chromium

Launch

Critical

Android/Termux

Chromium

CDP

Critical

Android/Termux

Brave

CDP

Critical

Android/Termux

Brave

Existing Browser

Medium

Android

Chrome

CDP

Optional

Android/Termux

Auto

Auto

High

47. Functional Testing

Every supported browser should pass:

☐ Browser detected

☐ Browser starts

☐ WebDriver connects

- ☐ CDP connects where required
- ☐ Eclipse RPG loads
- ☐ Login page loads
- ☐ Login works
- ☐ Login session persists where expected
- ☐ Main menu works
- ☐ Training works
- ☐ Capture works
- ☐ Mining works
- ☐ Shop automation works
- ☐ JavaScript works
- ☐ CSS selectors work
- ☐ XPath works
- ☐ WebDriverWait works
- ☐ Page navigation works
- ☐ Browser refresh works
- ☐ Extended execution works
- ☐ Browser disconnect recovery works
- ☐ Bot shuts down correctly

48. Regression Testing

Every browser architecture change must verify that existing Windows functionality still works.

Minimum regression:

Login
Training
Capture
Mining
Shop
Settings
Auto-update
Browser shutdown

A change that fixes Android but breaks Windows is not considered complete.

49. Security

Never commit:

Eclipse passwords

GitHub tokens

API keys

Cookies

Session databases

Browser credentials

Private browser profiles

Review .gitignore.

Browser profiles containing authentication data should never be committed.

50. Browser Credential Handling

The browser manager should never receive or store Eclipse credentials unnecessarily.

Authentication remains the responsibility of:

login.py

The browser manager should only provide:

driver

51. Logging Architecture

Logs should include environment information.

Example:

Platform: Android / Termux

Browser: Brave

Mode: CDP

Browser Version: ...

Selenium Version: ...

This makes support significantly easier.

52. Diagnostic Information

When an error occurs, logs should identify:

Platform
Browser
Browser version
Driver version
Selenium version
Connection mode
Debugger address
Relevant exception

Avoid exposing sensitive credentials.

53. Browser Compatibility Layer

The final browser manager should conceptually provide:

BrowserManager.detect_platform()
BrowserManager.detect_browsers()
BrowserManager.create()
BrowserManager.attach()
BrowserManager.test()
BrowserManager.close()

The exact API should be determined after the repository audit.

54. Fallback Rules

Recommended default:

Browser = Auto
Platform = Auto
Fallback = Disabled

If automatic detection is enabled:

Try preferred browser
↓
Try supported alternative
↓
Report failure

If the user explicitly chooses Brave:

Brave unavailable

↓
DO NOT silently switch
↓
Ask user whether to use another browser

55. Browser Health Checks

Before beginning major automation:

Browser health check
↓
Eclipse RPG reachable
↓
Driver responsive
↓
Session valid
↓
Begin automation

During long-running operation, periodically verify that the driver remains responsive.

56. Browser Disconnect Handling

Detect:

Invalid session ID

Connection refused

Browser closed unexpectedly

CDP connection lost

Driver timeout

Then attempt controlled recovery.

57. Android-Specific Browser Lifecycle

Android browsers may be controlled by the operating system.

Therefore the Android backend must account for:

Browser opened manually

Browser debugger restored

The bot should fail gracefully rather than corrupt its state.

58. Termux User Experience

The user should eventually be able to do something close to:

```
python main.py
```

The setup system should guide the user through anything that cannot be automated.

59. Termux Browser Setup Wizard

Potential future menu:

```
TERMUX BROWSER SETUP
Configure Android browser

Platform: Android / Termux

Browser detection:
✓ Chromium
✓ Brave

[1] Configure Chromium
[2] Configure Brave
[3] Test Chromium
[4] Test Brave
[5] Configure CDP
[6] Back
```

60. Browser Profiles Per Browser

Avoid sharing the same profile between:

Brave
Chrome
Chromium

unless the browser explicitly supports it safely.

Use browser-specific RPGBot profile directories.

Example:

```
profiles/  
  brave/  
  chrome/  
  chromium/
```

On Android, paths should be platform-appropriate.

61. Resource Usage

Android is more constrained than the Windows PC.

Measure:

RAM
CPU
Storage
Battery
Network usage

The bot should avoid unnecessary browser tabs/windows/processes.

62. Headless Mode

Support headless mode only where the selected browser/platform supports it reliably.

Default:

Headless = Disabled

Android may have different limitations than Windows.

Do not assume desktop headless behavior will work identically on Android.

63. Browser Window Management

Where applicable:

Reuse one browser

Reuse one tab

Avoid opening unnecessary tabs

Close temporary pages

Prevent runaway browser processes

64. Eclipse Website Compatibility

The website is the ultimate compatibility target.

Testing should verify that Eclipse RPG behaves correctly regardless of browser.

Pay particular attention to:

JavaScript

Dynamic elements

AJAX requests

Forms

Buttons

Cookies

Local storage

Page redirects

Session expiration

65. Mobile Browser Differences

Do not assume desktop browser behavior equals Android browser behavior.

Test:

Mobile viewport

Touch-oriented UI where applicable

Responsive page layout

Different DOM behavior

Different browser user agent

Popup behavior

Downloads

Navigation

JavaScript dialogs

The Selenium selectors should remain as browser-independent as possible.

66. User-Agent Strategy

Do not spoof a desktop user agent unless required.

First test Eclipse RPG normally.

If the website behaves differently on mobile, determine whether:

responsive layout

or:

user-agent detection

is responsible.

Only then introduce a controlled user-agent option.

67. Timeouts

Browser and network timeout settings should remain centralized.

Use the existing RPGBot timing configuration where possible.

Avoid hardcoding different timeout values throughout modules.

68. Existing RPGBot Timing Settings

Browser startup should integrate with existing:

WAIT_SHORT
WAIT_MEDIUM
WAIT_LONG

or equivalent settings.

Do not create multiple competing timeout systems.

69. Testing Browser Startup Without Eclipse

The Browser Test should work without requiring a successful Eclipse login.

It should verify:

Browser
Driver
CDP
Navigation
JavaScript
Element interaction

Then the actual Eclipse test verifies website compatibility.

70. Test Automation

Where practical, create automated tests for:

Platform detection
Browser configuration parsing
Browser selection
Path detection
Settings migration
Fallback logic

Browser integration tests can remain separate because they require actual browser installations.

71. Configuration Migration

If browser settings are added to an existing settings file:

Existing settings



Migration/default loader



New browser settings

No destructive rewrite of the user's existing settings should occur.

72. Configuration Validation

Reject invalid configurations such as:

Browser = "banana"

Platform = "Nintendo"

Mode = "whatever"

Report the problem and fall back to safe defaults.

73. Safe Defaults

Recommended:

Browser: Auto

Platform: Auto

Mode: Auto

Headless: No

Remote Debugging: No

Keep Browser Open: No

Fallback: No

74. Developer Debug Mode

Optional future setting:

Developer Browser Diagnostics

When enabled, provide more detailed startup information.

Normal users should receive concise output.

75. Architecture Boundaries

BrowserManager knows about:

Operating systems

Browsers

Drivers

Selenium

CDP

ADB

Browser profiles

Eclipse automation knows about:

Eclipse RPG

Pages

Selectors

Buttons

Forms

Game logic

Settings knows about:

Configuration

Persistence

Defaults

Validation

Update system knows about:

Versions

GitHub

Downloads

Updates

Restarting

Keep these responsibilities separate.

76. Avoid a Giant Browser Framework

The goal is not to build a new browser automation framework.

The goal is simply:

Centralize browser startup

+

Centralize platform differences

+

Keep Eclipse automation shared

Avoid unnecessary abstraction.

77. Final Repository Architecture

Possible final structure:

```
rpgbot/
├── browser/
│   ├── __init__.py
│   ├── manager.py
│   ├── platform.py
│   ├── windows.py
│   ├── android.py
│   ├── brave.py
│   ├── chrome.py
│   └── chromium.py
├── menus/
├── docs/
│   ├── TERMUX_SETUP.md
│   ├── ANDROID_BROWSER_SETUP.md
│   └── BROWSER_CONFIGURATION.md
├── scripts/
│   └── setup_termux.sh
├── config.py
├── settings.py
├── settings.json
├── login.py
├── training.py
└── capture.py
```

```
|— mining.py
|— shop.py
|— ...
```

This is an architectural target, not a requirement to create every file immediately.

78. Definition of Done — Windows

Windows is complete when:

- ☐ Brave works
- ☐ Chrome works
- ☐ Chromium works
- ☐ Auto detection works
- ☐ Existing RPGBot behavior remains intact
- ☐ Login works
- ☐ Training works
- ☐ Capture works
- ☐ Mining works
- ☐ Shop works
- ☐ Settings work
- ☐ Auto-update works
- ☐ Browser diagnostics work

79. Definition of Done — Android / Termux

Android is complete when:

- ☐ Termux setup documented
- ☐ Python works
- ☐ Selenium works
- ☐ Chromium works
- ☐ Brave works
- ☐ Browser detection works
- ☐ CDP works where required
- ☐ Existing-browser attachment works where supported
- ☐ Eclipse RPG loads
- ☐ Login works
- ☐ Training works
- ☐ Capture works
- ☐ Mining works
- ☐ Shop works
- ☐ Long-running operation is stable
- ☐ Browser recovery works

- [] Update system works
- [] Restart behavior works

80. Definition of Done — Cross-Platform

The overall project is complete when the same repository can run:

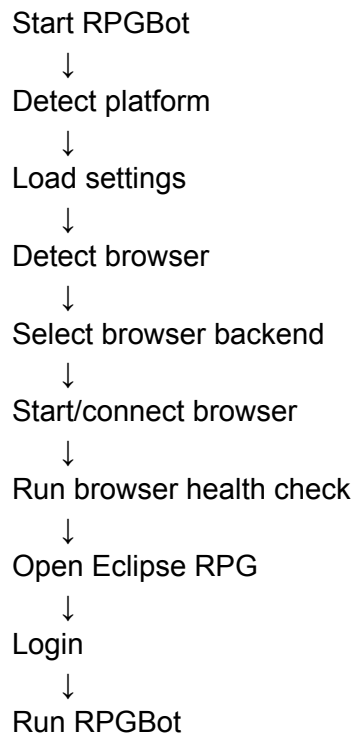
Windows + Brave
Windows + Chrome
Windows + Chromium

Android + Termux + Chromium
Android + Termux + Brave

using the same Eclipse automation modules.

81. Final User Experience

The ideal startup experience is:



Windows:

Windows
↓

BrowserManager



Brave / Chrome / Chromium



Selenium



Eclipse RPG

Android:

Android



Termux



BrowserManager



Chromium / Brave



Selenium / CDP



Eclipse RPG

82. Recommended First Coding Task

The first actual code change should be nothing more than an audit.

Find:

Where is WebDriver created?

Where are browser options created?

Where is ChromeDriver configured?

Where are browser paths defined?

Where are profiles defined?

Where are browsers closed?

Then document the findings.

Do not start with:

Termux

Brave

Android

CDP

until the current Windows browser implementation is understood.

83. Recommended Implementation Sequence

The safest complete sequence is:

1. Audit current browser startup
↓
2. Centralize Windows WebDriver creation
↓
3. Verify Windows regression
↓
4. Add Windows Brave
↓
5. Add Windows Chrome/Chromium
↓
6. Add browser settings
↓
7. Add diagnostics
↓
8. Prove Termux + Chromium independently
↓
9. Integrate Android Chromium
↓
10. Prove Android Brave independently
↓
11. Integrate Android Brave
↓
12. Add CDP/existing-browser support
↓
13. Make paths portable
↓
14. Add browser/session profiles
↓
15. Add browser recovery
↓
16. Make updater platform-aware
↓
17. Test long-running Termux operation
↓
18. Document everything
↓
19. Final cross-platform regression test

84. Guiding Rules

Throughout the project:

Rule 1

Do not break the existing Windows bot to gain Android support.

Rule 2

Do not duplicate Eclipse automation logic.

Rule 3

Browser-specific code belongs in the browser layer.

Rule 4

Platform-specific code belongs in platform backends.

Rule 5

Configuration belongs in the existing settings system.

Rule 6

Use branches and checkpoints for major changes.

Rule 7

Test every browser before declaring it supported.

Rule 8

Do not assume Android behaves like desktop Windows.

Rule 9

Do not assume Brave behaves exactly like Chromium simply because Brave is Chromium-based.

Rule 10

Do not hardcode device-specific Android paths or package IDs without detecting/confirming them.

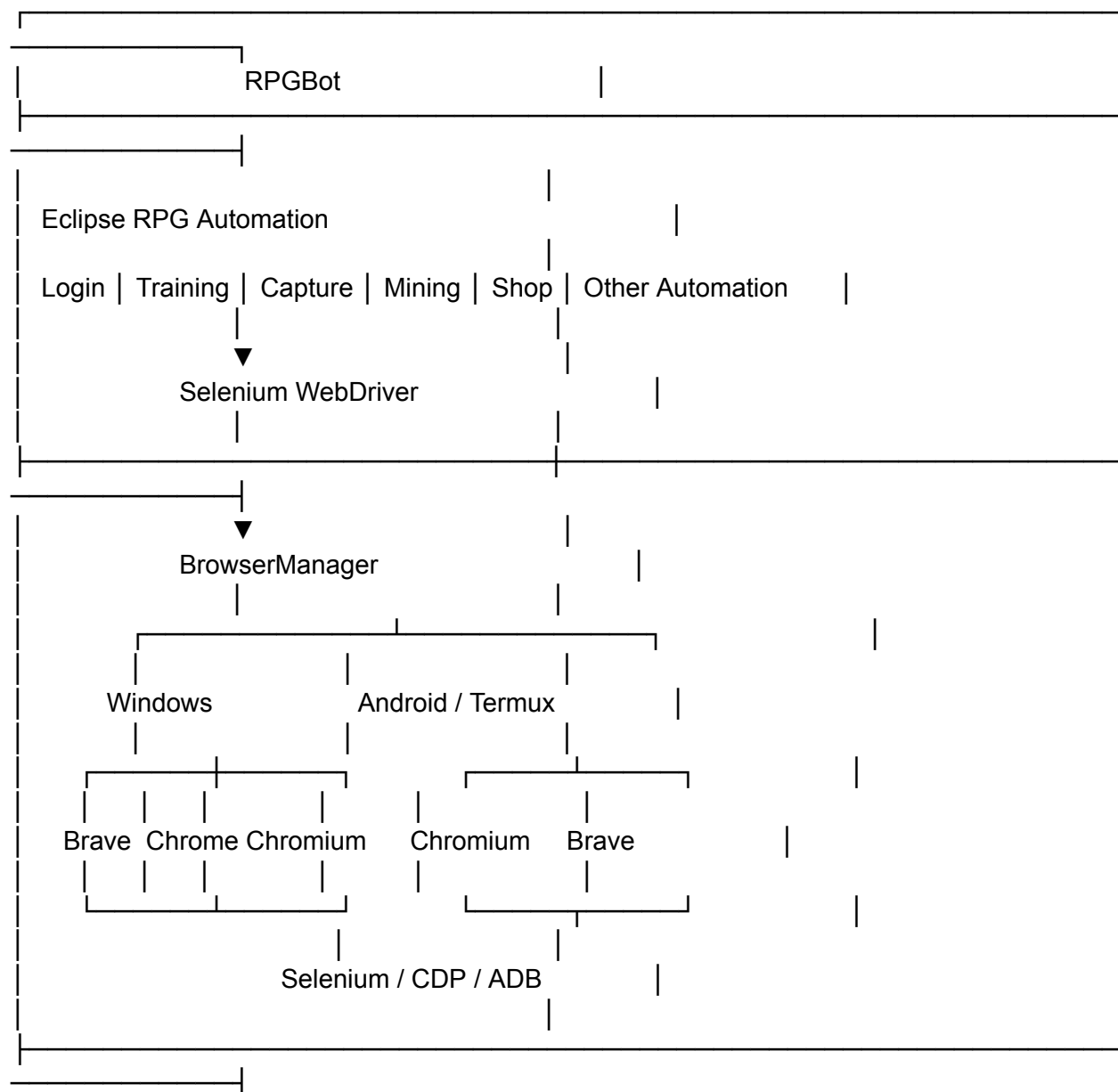
Rule 11

Keep credentials and browser session data out of Git.

Rule 12

Prefer small, reversible changes over a complete rewrite.

85. Final Architecture



Settings | Diagnostics | Profiles | Recovery | Updates | Restart |

86. Ultimate Goal

The completed system should make the operating system and browser largely invisible to the Eclipse automation.

The bot should not care whether it is running on:

Windows 11 + Brave

Windows 11 + Chrome

Windows 11 + Chromium

Android + Termux + Chromium

Android + Termux + Brave

The Eclipse RPG automation remains one shared system.

Only the browser/platform layer changes.

Core principle

One bot. One repository. One Eclipse automation codebase. Multiple browser and platform backends.